Univerzitet u Zenici
Politehnički fakultet

BILJEŠKE S PREDAVANJA · SEDMICA 5

DNS, e-pošta i SFTP kao kod

Imenovanje i komunikacija kao kod: DNS zone i zapisi u BIND-u (primarni i sekundarni server), zapisi kao Terraform resursi, e-pošta s Postfixom i Dovecotom uz TLS, SPF, DKIM i DMARC, te SFTP s chroot korisnicima iza vatrozida. Svaki servis se opisuje datotekama, a ishod se provjerava testom.

PREDMET	Računarstvo u oblaku
STUDIJ	II ciklus studija, III semestar, akademska godina 2026/27
NASTAVNIK	V. prof. dr. Đulaga Hadžić
ASISTENT	ass. Mr. Adin Jahić
TERMIN	petak, 13.11.2026, 17:15–20:15, Kabinet 3204
TEMA	DNS, e-pošta i SFTP kao kod
KOD	github.com/Dadyze/RuO-2026-27 (mape `s5/p1` do `s5/p4` i `s5/lab.sh`)

OKVIR NASTAVNE JEDINICE

Ciljevi i način rada

U sedmici 4 je mreža odredila *ko s kim smije razgovarati*. Ova sedmica uvodi servise preko kojih ljudi i programi stvarno komuniciraju: DNS pretvara imena u adrese, e-pošta prenosi poruke između domena, a SFTP prenosi datoteke. Sva tri servisa su stari i dobro poznati, ali su i čest izvor problema: zaboravljen serial u DNS zoni, mail server koji prosljeđuje tuđu poštu (otvoreni relej) ili SFTP korisnik koji vidi cijeli disk servera. Kad je konfiguracija kod, takve greške se vide u pregledu promjene i hvataju testom.

Za rad nije potreban račun u oblaku ni domena. Svaki server radi u svom **network namespace-u** (kao u sedmici 4), na zajedničkom virtualnom switchu, a pravi programi (BIND, Postfix, Dovecot, OpenSSH) se pokreću s konfiguracijom iz repozitorija. Domena laba je `ruo.lab` i postoji samo u labu.

Nakon nastavne jedinice student je u stanju:

- objasniti razliku između autoritativnog i rekurzivnog DNS servera, napisati zonsku datoteku i provjeriti je alatima `named-checkconf` i `named-checkzone`,
- objasniti ulogu SOA seriala, NOTIFY poruke i prijenosa zone između primarnog i sekundarnog servera,
- upravljati DNS zapisima kao Terraform resursima (RFC 2136, TSIG) i prepoznati drift,
- postaviti mail server koji prima poštu za domenu, nije otvoreni relej, traži TLS i prijavu, i potpisuje poruke DKIM-om,

- objasniti SPF, DKIM i DMARC i provjeriti ih na stvarnoj poruci,
- generisati SFTP server s chroot korisnicima, ključevima i vatrozidom iz podataka (Ansible rola) i provjeriti idempotentnost.

Okruženje i pokretanje

Alate za ovu sedmicu (BIND, Postfix, Dovecot, OpenDKIM, swaks, OpenSSH server) instalira `00-setup.sh`. Sistemske instance tih servisa skripta gasi, jer se servisi pokreću samo u labu. Ako je skripta već pokrenuta ranije, ponovo se pokrene (idempotentna je):

bash — Isječak 0 · početak

```
cd ~/RuO-2026-27 && git pull
./00-setup.sh                    # dodaje bind9, postfix, dovecot, opendkim, swaks, openssh-
server
make check                       # named, dig, postfix, dovecot, opendkim, swaks, sshd moraju
biti "ok"
make s5-p1                       # provjera DNS konfiguracije; zatim s5-p2 ... s5-p4,
opcionalni koraci: -up, -apply
```

RAD BEZ TROŠKOVA: PROVJERA UMJESTO IZVRŠAVANJA

Osnovni cilj svakog primjera samo provjerava kod: `named-checkconf` / `named-checkzone`, `terraform plan`, `postconf` / `doveconf` i `ansible-playbook --check --diff`. Stvarno pokretanje servisa ima sufiks `-up` ili `-apply`, traži `sudo` i dira samo lab: namespace-ove `ruo5-*`, direktorije `/etc/ruo5`, `/var/lib/ruo5`, `/var/lib/bind/ruo5` i `/srv/ruo5-sftp`, te dva lab korisnika (`ruo-ana`, `ruo-ben`). Sve se briše sa `make s5-clean`.

REDOSLIJED

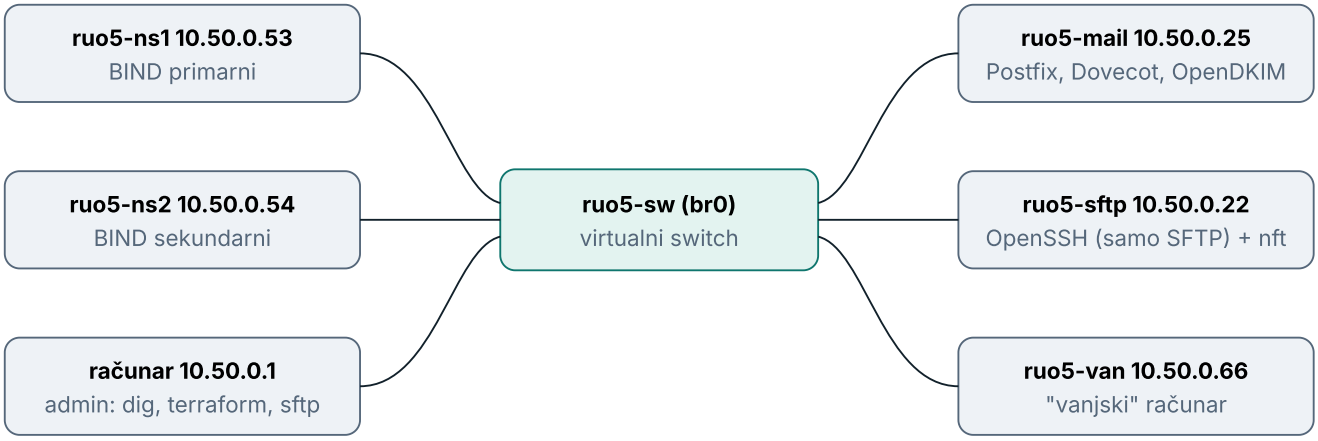
Primjeri 2 i 3 koriste DNS iz primjera 1 (Terraform piše u njegovu zonu, mail objavljuje DKIM ključ i oslanja se na MX, SPF i PTR zapise). Ciljevi `s5-p2-apply` i `s5-p3-up` zato sami podižu lab i DNS ako ne rade. Primjer 4 treba samo lab.

---

1 · POJAM I POZADINA

Lab: pet servera na jednom switchu

Lab (`s5/lab.sh`) pravi namespace `ruo5-sw` s bridgeom `br0`: to je virtualni switch. Na njega su veth parovima spojeni serveri i sam računar, koji ima adresu `10.50.0.1` i služi kao administratorska mašina (odavde rade `dig`, `terraform` i `sftp`). Namespace `ruo5-van` glumi "vanjski" računar: korisnika na laptopu ili tuđi server, pa služi i za negativne testove.



Slika 1. Topologija laba sedmice 5. Svi serveri koriste `10.50.0.53` kao resolver (datoteka `/etc/netns/<ns>/resolv.conf`), pa imena iz zone `ruo.lab` rade unutar laba bez ikakve izmjene na računaru.

> **POZADINA: /ETC/NETNS**
> Naredba `ip netns exec ns-ime program` pored mreže mijenja i pogled na `/etc`: datoteka `/etc/netns/ns-ime/resolv.conf` se za taj program montira preko `/etc/resolv.conf`. Tako svaki server dobija vlastiti DNS resolver, a računar zadržava svoj. Isti mehanizam se mogao koristiti za cijele direktorije (npr. `/etc/postfix`), ali primjeri radije zadaju putanju konfiguracije eksplicitno (`named -c`, `postfix -c`, `dovecot -c`, `sshd -f`), što radi i na pravom serveru.

---

2 · POJAM I POZADINA

DNS: zone, zapisi i ko odgovara

DNS je distribuirana baza: niko nema cijelu bazu, nego je prostor imena podijeljen na **zone**, a za svaku zonu odgovaraju njeni **autoritativni** serveri. Korijen (`.`) zna ko odgovara za `ba.`, ovaj zna ko odgovara za `unze.ba.`, i tako dalje: to je **delegacija**, a ostvaruje se NS zapisima. Klijenti ne pitaju autoritativne servere direktno, nego **rekurzivni resolver** (onaj iz `/etc/resolv.conf`, npr. ruter, ISP ili `1.1.1.1`) koji prolazi lancem delegacija i pamti odgovore onoliko dugo koliko kaže **TTL**.

ZAPIS	ZNAČENJE	PRIMJER IZ ZONE RUO.LAB
`SOA`	početak zone: glavni server, kontakt, **serial** i tajmeri sekundarnih	`ns1.ruo.lab. hostmaster.ruo.lab. 2026111301 ...`
`NS`	autoritativni serveri zone ili poddomene (delegacija)	`@ NS ns1`, `oblak NS ns1`
`A` / `AAAA`	ime → IPv4 / IPv6 adresa	`mail A 10.50.0.25`
`CNAME`	ime je drugo ime za drugo ime (alias)	`imap CNAME mail`
`MX`	koji server prima poštu za domenu (s prioritetom)	`@ MX 10 mail`
`TXT`	slobodan tekst; nosi SPF, DKIM i DMARC politike	`@ TXT "v=spf1 mx -all"`
`SRV`	gdje je servis (port i host), npr. za automatsko podešavanje klijenata	`_submission._tcp SRV 0 1 587 mail`
`PTR`	adresa → ime (reverzna zona `in-addr.arpa`)	`25 PTR mail.ruo.lab.`
`CAA`	koji CA smije izdati TLS certifikat za domenu	`@ CAA 0 issue "letsencrypt.org"`

> **POZADINA: PRIMARNI, SEKUNDARNI I SERIAL**
> Zona se drži na najmanje dva servera, na različitim mrežama, jer bez DNS-a ne radi ništa drugo. **Primarni** ima izvor zone (datoteku ili bazu), a **sekundarni** drži kopiju koju preuzima prijenosom zone (AXFR cijela, IXFR samo razlike). Sekundarni preuzima zonu *samo ako je serial u SOA zapisu veći* od onog koji već ima. Nakon promjene primarni šalje **NOTIFY**, sekundarni uporedi serial i, ako je veći, povuče promjene; ako NOTIFY ne stigne, provjerava sam svakih `refresh` sekundi. Zaboravljen serial je zato klasična greška: primarni odgovara novim podacima, sekundarni starim, a klijent dobija jedan ili drugi odgovor zavisno od toga koga je pitao.

U oblaku se autoritativni DNS najčešće kupuje kao usluga (AWS Route 53, Azure DNS, Google Cloud DNS, Cloudflare): zona je resurs, a zapisi se mijenjaju kroz API. Na sopstvenim serverima se koriste **BIND** (najstariji i najpotpuniji), **Knot** i **PowerDNS**, a **CoreDNS** je DNS server od dodataka (plugin) koji je i zadani DNS u Kubernetesu (vidi napomenu u primjeru 1). Princip je svugdje isti: zona je kod ili podaci, a promjena prolazi provjeru prije objave.

3 · PRIMJER 1 · S5/P1

BIND: zona kao kod, primarni i sekundarni server

> **POZADINA PRIMJERA**
> Zona `ruo.lab` je obična tekstualna datoteka u Gitu. Primarni server (`ruo5-ns1`) je učitava, a sekundarni (`ruo5-ns2`) nema vlastitih zonskih datoteka: sve preuzima od primarnog. Server je **samo autoritativni** (`recursion no`): odgovara za svoje zone, a tuđe domene odbija, što sprječava zloupotrebu servera za DNS amplifikacijske napade. Prijenos zone (AXFR) dozvoljen je samo sekundarnom, jer cijela zona otkriva sva imena i adrese u mreži.

dns zone — Isječak 1 · s5/p1/zone/db.ruo.lab (izvod)

```
$TTL 300
$ORIGIN ruo.lab.
@         IN SOA  ns1.ruo.lab. hostmaster.ruo.lab. (
                  2026111301 ; serial
                  3600       ; refresh: koliko često sekundarni provjerava serial
                  600        ; retry: ponovni pokušaj ako primarni ne odgovara
                  1209600    ; expire: sekundarni prestaje odgovarati ako 14 dana nema
primarnog
                  300 )      ; negativni TTL: koliko se pamti odgovor "ne postoji" (NXDOMAIN)

@         IN NS     ns1
@         IN NS     ns2
ns1       IN A      10.50.0.53
ns2       IN A      10.50.0.54

mail      IN A      10.50.0.25
sftp      IN A      10.50.0.22
imap      IN CNAME  mail

@         IN MX     10 mail
@         IN TXT    "v=spf1 mx -all"
_dmarc    IN TXT    "v=DMARC1; p=reject; adkim=s; aspf=s; rua=mailto:dmarc@ruo.lab"
_submission._tcp IN SRV 0 1 587 mail
@         IN CAA    0 issue "letsencrypt.org"

; delegacije dinamičkim zonama (zapise u njima mijenjaju alati, ne ova datoteka)
oblak       IN NS   ns1
oblak       IN NS   ns2
_domainkey  IN NS   ns1
_domainkey  IN NS   ns2
```

Ime bez tačke na kraju je relativno (`mail` znači `mail.ruo.lab.`), a `@` je sama zona. Zapisi `ns1` i `ns2` tipa A su *glue*: bez njih bi resolver, da bi našao `ns1.ruo.lab`, morao pitati upravo server čiju adresu traži. Zona ima i reverzni par (`db.10.50.0`): mail serveri provjeravaju da se ime i adresa pošiljaoca slažu u oba smjera.

named.conf — Isječak 2 · s5/p1/named-primarni.conf (izvod)

```
include "keys.conf";                // TSIG ključevi: generišu se pri pokretanju, nisu u Gitu

options {
  listen-on { 10.50.0.53; 127.0.0.1; };
```

```
  recursion no;                       // samo autoritativni odgovori
  allow-transfer { 10.50.0.54; };     // kopiju cijele zone (AXFR) smije uzeti samo sekundarni
  notify yes;                         // nakon promjene javi sekundarnom (NOTIFY)
};

// statičke zone: datoteka je u Gitu, mijenja se commitom i povećanjem seriala
zone "ruo.lab"              { type primary; file "zone/db.ruo.lab"; };
zone "0.50.10.in-addr.arpa" { type primary; file "zone/db.10.50.0"; };

// dinamičke zone: zapise mijenjaju alati (RFC 2136), svaki ključ samo svoju zonu
zone "oblak.ruo.lab" {
  type primary; file "dyn/db.oblak.ruo.lab";
  update-policy { grant ruo-tf zonesub ANY; };          // Terraform (s5/p2)
};
zone "_domainkey.ruo.lab" {
  type primary; file "dyn/db._domainkey.ruo.lab";
  update-policy { grant ruo-dkim zonesub TXT; };        // DKIM ključ (s5/p3), samo TXT
};
```

STATIČKE I DINAMIČKE ZONE

Glavna zona je statička: mijenja se samo kroz Git, pa svaka izmjena ima autora, opis i pregled. Zapise koje pravi automatika (Terraform, objava DKIM ključa, ACME izazovi) ima smisla odvojiti u posebne, delegirane zone koje se mijenjaju dinamičkim UPDATE-om. Tako glavna datoteka nikad ne dobije zapis koji nije prošao pregled, a svaki alat ima ključ koji smije mijenjati samo svoju zonu i samo potrebne tipove zapisa (načelo najmanjih prava, kao kod security grupa u sedmici 4).

bash — Isječak 3 · pokretanje

```
make s5-p1           # named-checkconf -z i named-checkzone (bez root-a, ništa se ne pokreće)
make s5-p1-up        # opcionalno: lab, primarni i sekundarni server, dig upiti (traži sudo)
make s5-p1-serial    # opcionalno: izmjena zone bez i sa povećanjem seriala
```

izlaz make s5-p1: provjera bez pokretanja servera

```
== named-checkconf -z: sintaksa konfiguracije i učitavanje svih zona primarnog
zone ruo.lab/IN: loaded serial 2026111301
zone 0.50.10.in-addr.arpa/IN: loaded serial 2026111301
zone oblak.ruo.lab/IN: loaded serial 1
zone _domainkey.ruo.lab/IN: loaded serial 1

== named-checkconf: sekundarni
named-sekundarni.conf je ispravan
```

U odgovoru na upit, zastavica `aa` (*authoritative answer*) znači da je odgovorio server koji je izvor podataka, a ne neki keš. Upozorenje `recursion requested but not available` je očekivano: `dig` traži rekurziju, a server je namjerno nema.

izlaz make s5-p1-up (skraćeno)

```
== 1. Puni odgovor jednog upita: zaglavlje (flags: aa = autoritativni odgovor) i sekcije
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 6332
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available
;; ANSWER SECTION:
mail.ruo.lab.           300     IN      A       10.50.0.25

== 2. Zapisi zone (kratko): A, CNAME, MX, SRV, CAA
imap.ruo.lab.           IN      CNAME   mail.ruo.lab.
mail.ruo.lab.           IN      A       10.50.0.25
ruo.lab.                IN      MX      10 mail.ruo.lab.
_submission._tcp.ruo.lab. IN    SRV     0 1 587 mail.ruo.lab.
ruo.lab.                IN      CAA     0 issue "letsencrypt.org"

== 4. Reverzni upit (PTR): adresa -> ime
$ dig +short @10.50.0.53 -x 10.50.0.25
mail.ruo.lab.

== 5. Isti odgovor daje i sekundarni (isti serial u SOA)
$ dig +short @10.50.0.54 ruo.lab SOA
ns1.ruo.lab. hostmaster.ruo.lab. 2026111301 3600 600 1209600 300

== 6. Prijenos zone (AXFR): sekundarnom dozvoljen, svima ostalima odbijen
[ruo5-ns2]$ dig @10.50.0.53 ruo.lab AXFR | grep -c IN
21
[ruo5-van]$ dig @10.50.0.53 ruo.lab AXFR
; Transfer failed.

== 7. Nepostojeće ime (NXDOMAIN) i tuđa domena (REFUSED: server nije rekurzivni)
$ dig @10.50.0.53 nema.ruo.lab A   ->  status: NXDOMAIN
$ dig @10.50.0.53 example.com A   ->  status: REFUSED
```

Zašto serial mora rasti

Cilj `s5-p1-serial` dodaje zapis u radnu kopiju zone na primarnom i učitava je (`rndc reload`), prvo bez promjene seriala, pa s povećanjem. Ime zapisa sadrži vrijeme, pa se demonstracija može ponavljati.

izlaz make s5-p1-serial

```
== 1. Dodajemo zapis test164746.ruo.lab u radnu kopiju zone, ali serial OSTAJE 2026111301
$ rndc reload ruo.lab
zone reload queued
primarni:   10.50.0.99   (serial 2026111301)
sekundarni:              (serial 2026111301, prazno = ne zna za zapis)

== 2. Povećavamo serial (2026111301 -> 2026111302): primarni šalje NOTIFY, sekundarni povlači
promjenu
$ rndc reload ruo.lab
zone reload queued
primarni:   10.50.0.99   (serial 2026111302)
sekundarni: 10.50.0.99   (serial 2026111302)

== 3. Dnevnik sekundarnog: NOTIFY i prijenos (IXFR/AXFR)
 transfer of 'ruo.lab/IN' from 10.50.0.53#53: Transfer status: success
 transfer of 'ruo.lab/IN' from 10.50.0.53#53: Transfer completed: 1 messages, 22 records, ...
(serial 2026111302)
```

GREŠKE KOJE SE NAJČEŠĆE PRAVE

Zaboravljen serial (sekundarni ostaje na starim podacima). Zaboravljena tačka na kraju punog imena: `mail.ruo.lab` bez tačke postaje `mail.ruo.lab.ruo.lab.`. CNAME na imenu koje ima i druge zapise (npr. na `@`, gdje su SOA i NS): standard to zabranjuje, a `named-checkzone` to javlja. Smanjen serial: sekundarni ga ignoriše dok se ne izjednači, pa se serial nikad ne vraća unazad. Promjena TTL-a se ne vidi odmah: resolveri drže stari odgovor dok ne istekne stari TTL, zato se TTL prije planirane promjene adrese unaprijed smanjuje.

COREDNS UKRATKO

CoreDNS radi isti posao drugačijim pristupom: svaka funkcija je dodatak u datoteci `Corefile`. Ista zona bi se poslužila blokom `ruo.lab { file db.ruo.lab; transfer { to 10.50.0.54 } }`. U Kubernetesu (sedmica 3, `kind`) CoreDNS je zadani DNS klastera i dodatkom `kubernetes` pravi zapise za servise (`web.default.svc.cluster.local`) iz stanja klastera, što je DNS kao kod u najčistijem obliku: zapis postoji zato što postoji resurs.

---

4 · PRIMJER 2 · S5/P2

DNS zapisi kao Terraform resursi

POZADINA PRIMJERA

Provider `hashicorp/dns` mijenja zapise standardnim protokolom za dinamičke izmjene (**RFC 2136**, DNS UPDATE), a svaki zahtjev potpisuje **TSIG** ključem (HMAC-SHA256 nad porukom i vremenom). Server iz primjera 1 prihvata UPDATE za zonu `oblak.ruo.lab` samo potpisan ključem `ruo-tf`. Tajna ključa se generiše pri pokretanju laba i nije u Gitu: `make` je predaje Terraformu kroz varijablu okruženja `TF_VAR_tsig_secret`, a varijabla je označena `sensitive`, pa se ne ispisuje u planu.

hcl — Isječak 4 · s5/p2/main.tf (izvod)

```hcl
provider "dns" {
  update {
    server        = "10.50.0.53"
    key_name      = "ruo-tf."
    key_algorithm = "hmac-sha256"
    key_secret    = var.tsig_secret
  }
}

# servisi iz sedmice 4 (segmenti web/app/db) dobijaju imena; dvije adrese = dva A zapisa
(round-robin)
variable "servisi" {
  type = map(list(string))
  default = {
    web = ["10.40.1.10", "10.40.1.11"]
    app = ["10.40.2.10"]
    db  = ["10.40.3.10"]
  }
}

resource "dns_a_record_set" "servis" {
  for_each  = var.servisi
  zone      = var.zona                    # "oblak.ruo.lab."
```

```
  name      = each.key
  addresses = each.value
  ttl       = 300
}

resource "dns_cname_record" "www" {
  zone  = var.zona
  name  = "www"
  cname = "web.${var.zona}"
  ttl   = 300
}
```

bash — Isječak 5 · pokretanje

```
make s5-p2          # init, validate i plan bez kontakta sa serverom (-refresh=false)
make s5-p2-apply    # opcionalno: apply, ponovni apply, drift i ispravak (podiže lab i DNS ako ne rade)
make s5-p2-clean    # destroy (ako DNS radi) i brisanje statea
```

izlaz make s5-p2-apply: koraci 1 i 2

```
== 1. apply: zapisi u zoni oblak.ruo.lab
Apply complete! Resources: 5 added, 0 changed, 0 destroyed.
web.oblak.ruo.lab      10.40.1.10 10.40.1.11
app.oblak.ruo.lab      10.40.2.10
db.oblak.ruo.lab       10.40.3.10
www.oblak.ruo.lab      web.oblak.ruo.lab. 10.40.1.10 10.40.1.11

== 2. apply ponovo: ocekujemo 0 added / 0 changed = idempotentnost
No changes. Your infrastructure matches the configuration.
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
```

Odgovore daje *sekundarni* server (`@10.50.0.54`): UPDATE je stigao na primarni, a NOTIFY je promjenu odmah prenio dalje. Drift se pravi kao u sedmici 2, ali "ručna" izmjena je ovdje `nsupdate`: neko briše zapis `app` i mijenja adresu za `db` mimo koda. Plan pokazuje tačno to, a apply vraća stanje iz koda:

izlaz make s5-p2-apply: koraci 3 i 4 (drift)

```
== 3. DRIFT: zapis mijenjamo mimo Terraforma (nsupdate)
$ nsupdate -k ruo-tf.key   (briše app.oblak.ruo.lab i mijenja adresu za db)
app: ''   db: 10.40.3.99

  # dns_a_record_set.servis["app"] will be created
  # dns_a_record_set.servis["db"] will be updated in-place
  ~ resource "dns_a_record_set" "servis" {
      ~ addresses = [
          - "10.40.3.99",
          + "10.40.3.10",
        ]
Plan: 1 to add, 1 to change, 0 to destroy.

== 4. apply vraca zapise na stanje iz koda
Apply complete! Resources: 1 added, 1 changed, 0 destroyed.
10.40.2.10
10.40.3.10
```

GRANICA: TERRAFORM VIDI SAMO SVOJE

Zapis koji neko *doda* ručno, a nije u kodu, Terraform ne primjećuje: state sadrži samo resurse koje je sam napravio. Zato se dinamičke zone daju samo jednom vlasniku (jedan ključ, jedan alat), a za potpunu kontrolu se zona periodično poredi s očekivanim stanjem (npr. `dig AXFR` i poređenje s izlazom `terraform state list`).

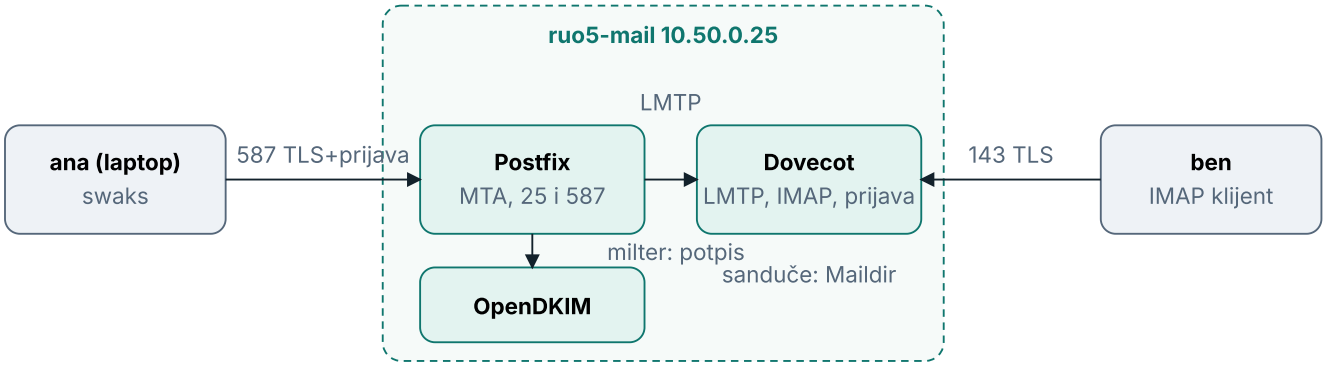
ISTI KOD U OBLAKU

Za Route 53 bi se promijenili samo provider i tip resursa, a struktura ostaje: `resource "aws_route53_record" "web" { zone_id = aws_route53_zone.z.zone_id, name = "web", type = "A", ttl = 300, records = ["10.40.1.10"] }`. Isto vrijedi za `azurerm_dns_a_record` i `cloudflare_record`. Lokalni BIND s RFC 2136 je besplatan način da se vježba identičan tok rada: plan, apply, state, drift.

5 · POJAM I POZADINA

E-pošta: put poruke i tri provjere pošiljaoca

Poruka prolazi kroz više programa. Korisnik je predaje svom serveru preko **submission** porta (587, STARTTLS i obavezna prijava). **MTA** (Postfix) je prosljeđuje serveru primaoca, kojeg nalazi preko **MX** zapisa domene, na port 25. Server primaoca je predaje **MDA**-u (Dovecot, preko LMTP-a) koji je spremi u sanduče, a primalac je čita preko **IMAP**-a (143 sa STARTTLS ili 993 s TLS-om). U labu su pošiljalac i primalac na istom serveru, pa se korak "server na server" svodi na isporuku u lokalno sanduče.



Slika 2. Put poruke u labu. Postfix prijavu korisnika provjerava preko Dovecota (SASL), pa postoji jedna baza korisnika za slanje i čitanje; OpenDKIM je *milter*, filter kroz koji Postfix propušta svaku poruku.

SMTP je nastao kad su se svi serveri poznavali: polje `From:` može napisati bilo ko. Zato primalac danas provjerava pošiljaoca kroz tri mehanizma, a sva tri su **DNS zapisi** domene pošiljaoca, što ovu temu veže uz primjer 1:

MEHANIZAM	PITANJE KOJE ODGOVARA	ZAPIS U LABU
SPF	Smije li ova IP adresa slati poštu u ime domene (adresa iz *envelope*, `MAIL FROM`)?	`ruo.lab TXT "v=spf1 mx -all"`: samo MX serveri; `-all` = sve ostalo odbij
DKIM	Da li je poruku potpisala domena i da li je neizmijenjena?	`ruo2026._domainkey.ruo.lab TXT "v=DKIM1; k=rsa; p=MIIB..."`: javni ključ
DMARC	Poklapa li se domena iz SPF-a ili DKIM-a s domenom iz `From:` (poravnanje), i šta raditi ako ne?	`_dmarc.ruo.lab TXT "v=DMARC1; p=reject; adkim=s; aspf=s; rua=..."`

> **POZADINA: ZAŠTO TREBA I DMARC**
> SPF i DKIM sami ne štite polje `From:` koje korisnik vidi. Napadač može proći SPF za svoju domenu (`MAIL FROM: x@napadac.com`) i potpisati poruku svojim DKIM ključem (`d=napadac.com`), a u `From:` napisati `direktor@ruo.lab`. DMARC traži da bar jedna od dvije provjere prođe *za istu domenu* koja je u `From:`. Politika `p=reject` kaže primaocu da takvu poruku odbije, a `rua` je adresa na koju primaoci šalju zbirne izvještaje. U praksi se kreće od `p=none` (samo izvještaji), pa kad izvještaji pokažu da sva legitimna pošta prolazi, prelazi na `quarantine` i `reject`. Od 2024. godine Gmail i Yahoo za masovne pošiljaoce traže sve tri provjere.

TLS I ACME

Certifikat u labu izdaje lokalni CA (`mail.sh` ga pravi naredbom `openssl`), a klijenti ga provjeravaju jer im je CA eksplicitno zadan. Na javnom serveru certifikat izdaje javni CA protokolom **ACME** (Let's Encrypt, `certbot`), koji prvo provjeri da podnosilac kontroliše domenu: *HTTP-01* (datoteka na web serveru) ili *DNS-01* (TXT zapis `_acme-challenge`). DNS-01 je jedini način za wildcard certifikat i za servere bez web servera, kakav je mail server. Dodatak `certbot-dns-rfc2136` taj TXT zapis upisuje istim mehanizmom kao Terraform u primjeru 2: dinamičkim UPDATE-om s TSIG ključem, idealno u posebnu, delegiranu zonu `_acme-challenge`. Zapis `CAA` u zoni pri tome ograničava koji CA uopće smije izdati certifikat.

---

6 · PRIMJER 3 · S5/P3

Postfix, Dovecot i OpenDKIM: mail server kao kod

POZADINA PRIMJERA

Konfiguracija je u četiri datoteke (`postfix/main.cf`, `postfix/master.cf`, `dovecot/dovecot.conf`, `opendkim.conf`) i jednoj datoteci s podacima (`korisnici.txt`). Skripta `mail.sh` ih postavlja u `/etc/ruo5/mail`, od podataka pravi popis sandučića za Postfix i hashirane lozinke za Dovecot, pravi TLS certifikat i DKIM ključ, objavljuje javni ključ u DNS-u i pokreće servise u namespace-u `ruo5-mail`. Instance su potpuno odvojene od eventualnog Postfixa ili Dovecota na računaru: vlastita konfiguracija, red čekanja, sandučići i `base_dir`.

postfix — Isječak 6 · s5/p3/postfix/main.cf (izvod)

```
myhostname = mail.ruo.lab
mydomain = ruo.lab

# primanje pošte za @ruo.lab: postojeći sandučići, predaja Dovecotu preko LMTP-a
virtual_mailbox_domains = ruo.lab
virtual_mailbox_maps = texthash:/etc/ruo5/mail/postfix/sanducici
virtual_transport = lmtp:unix:private/dovecot-lmtp

# TLS (certifikat laba; u produkciji ACME, npr. Let's Encrypt)
smtpd_tls_cert_file = /etc/ruo5/mail/tls/mail.crt
smtpd_tls_key_file = /etc/ruo5/mail/tls/mail.key
smtpd_tls_security_level = may

# prijava korisnika (SASL) ide preko Dovecota: jedna baza korisnika za slanje i čitanje
smtpd_sasl_type = dovecot
smtpd_sasl_path = private/auth

# NIJE otvoreni relej: tuđe domene samo za prijavljene korisnike
smtpd_relay_restrictions = permit_mynetworks, permit_sasl_authenticated,
reject_unauth_destination

# DKIM: OpenDKIM kao milter potpisuje odlaznu i provjerava dolaznu poštu
smtpd_milters = inet:127.0.0.1:8891
```

postfix — Isječak 7 · s5/p3/postfix/master.cf: submission (587)

```
submission inet  n       -       n       -       -       smtpd
  -o smtpd_tls_security_level=encrypt       # bez STARTTLS nema dalje
  -o smtpd_sasl_auth_enable=yes
  -o smtpd_tls_auth_only=yes                # lozinka se nudi tek nakon TLS-a
  -o smtpd_client_restrictions=permit_sasl_authenticated,reject
  -o milter_macro_daemon_name=ORIGINATING   # OpenDKIM: ovo je odlazna pošta, potpiši je
```

OTVORENI RELEJ

Najvažniji red konfiguracije je `reject_unauth_destination`: server prima poštu samo za svoje domene, a tuđe domene prosljeđuje samo prijavljenim korisnicima. Server bez tog pravila je *otvoreni relej*: za nekoliko sati ga pronađu spameri, a njegova adresa završi na crnim listama, nakon čega ni legitimna pošta ne prolazi. Test u ovom primjeru zato prvo provjerava upravo to.

DKIM ključ se pravi jednom (`opendkim-genkey`). Privatni dio ostaje na serveru (čita ga samo korisnik `opendkim`), a javni se objavljuje u zoni `_domainkey.ruo.lab` dinamičkim UPDATE-om s ključem `ruo-dkim`, koji smije mijenjati samo TXT zapise te zone:

bash — Isječak 8 · s5/p3/mail.sh: objava DKIM ključa (izvod)

```
opendkim-genkey -b 2048 -d ruo.lab -s ruo2026 -D /var/lib/ruo5/dkim
nsupdate -k /var/lib/ruo5/kljucevi/ruo-dkim.key <<EOF
server 10.50.0.53
zone _domainkey.ruo.lab
update delete ruo2026._domainkey.ruo.lab. TXT
update add ruo2026._domainkey.ruo.lab. 300 TXT "v=DKIM1; h=sha256; k=rsa; " "p=MIIBIjAN..."
send
EOF
```

Selektor (`ruo2026`) omogućava više ključeva istovremeno: pri zamjeni ključa novi selektor se objavi, server počne potpisivati novim, a stari zapis se ukloni tek kad sve poruke potpisane njime budu isporučene. TXT zapis duži od 255 znakova dijeli se u više navodnicima odvojenih dijelova, koje primalac spaja.

bash — Isječak 9 · pokretanje

```
make s5-p3          # postconf -n/-M, doveconf -n, opendkim -n (bez root-a, ništa se ne pokreće)
make s5-p3-up       # opcionalno: TLS, DKIM, servisi i test (podiže lab i DNS ako ne rade; traži sudo)
make s5-p3-clean    # gasi servise i briše sandučiće, ključ i certifikat
```

Test (`mail.sh test`) radi iz `ruo5-van`, kao korisnik koji nije na mreži servera. Prvo provjerava ono što ne smije raditi:

izlaz make s5-p3-up: koraci 2 i 3 (TLS i negativni slučajevi)

```
== 2. TLS: certifikat odgovara imenu i potpisan je CA-om laba (STARTTLS na 587)
  New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
  Verify return code: 0 (ok)
  ok: certifikat je ispravan za mail.ruo.lab

== 3. Negativni slučajevi: otvoreni relej i lozinka bez TLS-a
$ swaks --server mail.ruo.lab:25 --from neko@example.org --to zrtva@example.com   (iz ruo5-van,
bez prijave)
  <** 554 5.7.1 <zrtva@example.com>: Relay access denied
  ok: server odbija prosljeđivanje tuđe pošte (nije otvoreni relej)
$ swaks --server mail.ruo.lab:587 --auth ... (bez STARTTLS)
  *** Host did not advertise authentication
  ok: prijava bez TLS-a nije moguća
```

Zatim šalje poruku kao `ana@ruo.lab`, čita je IMAP-om kao `ben@ruo.lab` i provjerava potpis. U SMTP razgovoru `<-` označava nešifrovani dio, a `<~` dio nakon STARTTLS-a: lozinka (`235 Authentication successful`) je prošla tek kroz šifrovani kanal.

izlaz: koraci 4 do 7 (slanje, čitanje, DKIM, SPF, DMARC)

```
== 4. Slanje: ana@ruo.lab -> ben@ruo.lab preko submission (587, STARTTLS, prijava)
  <-  220 mail.ruo.lab ESMTP
  <-  220 2.0.0 Ready to start TLS
  <~  235 2.7.0 Authentication successful
  <~  250 2.0.0 Ok: queued as 9B68CF611F

== 5. Čitanje: IMAP (143, STARTTLS) kao ben@ruo.lab, zaglavlja zadnje poruke
  INBOX: 1 poruka, TLS: TLSv1.3
  DKIM-Signature: v=1; a=rsa-sha256; c=relaxed/simple; d=ruo.lab; s=ruo2026;
  From: ana@ruo.lab
  Subject: Pozdrav iz sedmice 5
  Received: from laptop (unknown [10.50.0.66])
        (using TLSv1.3 with cipher TLS_AES_256_GCM_SHA384 (256/256 bits)

== 6. DKIM: primalac provjerava potpis javnim ključem iz DNS-a
$ dkimverify < poruka.eml
signature ok
$ dkimverify < izmijenjena.eml   (promijenjen Subject)
signature verification failed

== 7. SPF i DMARC: smije li IP slati u ime ruo.lab, i da li se domene poklapaju s From
  SPF 10.50.0.25  mail.ruo.lab (MX)  -> pass
  SPF 10.50.0.66  tuđi server        -> fail
  DMARC: From domena=ruo.lab, DKIM d=ruo.lab (poravnato: True), DKIM=pass -> pass
```

`dkimverify` radi u `ruo5-van`, gdje nema privatnog ključa: javni ključ dohvata iz DNS-a (`ruo2026._domainkey.ruo.lab`), tačno kao pravi primalac. Izmjena jednog zaglavlja ruši potpis, jer potpis pokriva zaglavlja navedena u `h=` i hash tijela poruke. SPF provjera za `10.50.0.66` daje `fail` jer ta adresa nije MX domene. Na kraju test u dnevniku Postfixa pokazuje put poruke: prijava (`sasl_username=ana@ruo.lab`), `cleanup` (milter dodaje potpis) i isporuka preko LMTP-a (`status=sent`), te završava sa `Sve provjere su prošle.`

GRANICE LABA

Pravi mail server na internetu traži još: PTR zapis za javnu adresu (postavlja ga vlasnik IP adrese, tj. pružatelj), otvoren izlazni port 25 (većina oblaka ga zadano blokira zbog spama), DNSSEC i MTA-STS za zaštitu veze između servera, filtriranje spama i nadzor reputacije adrese. Zato organizacije poštu sve češće drže kod pružatelja (Microsoft 365, Google Workspace), a sami upravljaju samo DNS zapisima (MX, SPF, DKIM, DMARC), što je opet kod iz primjera 1 i 2.

7 · PRIMJER 4 · S5/P4

SFTP s chroot korisnicima iz podataka: Ansible rola

> **POZADINA PRIMJERA**
> SFTP je protokol za prijenos datoteka unutar SSH veze: šifrovan je, koristi SSH ključeve i ne treba posebne portove, za razliku od starog FTP-a (nešifrovane lozinke, odvojena podatkovna veza) i FTPS-a. Za korisnike koji samo razmjenjuju datoteke (partneri, skripte za backup) važno je ograničiti ih: `ForceCommand internal-sftp` ne dozvoljava shell, a `ChrootDirectory` korijen sesije pretvara u jedan direktorij, pa korisnik ne vidi ostatak diska. Chroot ima strogo pravilo: direktorij i svi roditelji moraju pripadati `root` -u i niko drugi ne smije pisati u njih, inače sshd odbija prijavu. Zato korisnik dobija poddirektorij `upload` u koji jedino smije pisati.

Rola koristi isti obrazac kao vatrozid iz sedmice 4: politika je u podacima, a šabloni od nje prave konfiguraciju. Novi korisnik je jedan red u `vars.yml`:

yaml — Isječak 10 · s5/p4/group_vars/sftp/vars.yml

```yaml
sftp_korisnici:
  - ime: ruo-ana
    kljuc: "{{ lookup('file', playbook_dir + '/kljucevi/ruo-ana.pub') }}"
  - ime: ruo-ben
    kljuc: "{{ lookup('file', playbook_dir + '/kljucevi/ruo-ben.pub') }}"

# vatrozid: SSH/SFTP smije samo admin mreža (računar 10.50.0.1), ostali se odbacuju
sftp_dozvoljeni: [10.50.0.1/32]

# U labu server radi u namespace-u ruo5-sftp (s5/lab.sh); na pravom serveru sftp_ns je prazan.
sftp_ns: ruo5-sftp
sftp_adresa: 10.50.0.22
```

jinja2 — Isječak 11 · s5/p4/roles/sftp/templates/sshd_config.j2 (izvod)

```jinja2
ListenAddress {{ sftp_adresa }}
HostKey {{ sftp_etc }}/ssh_host_ed25519_key
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
AuthorizedKeysFile {{ sftp_etc }}/kljucevi/%u
AllowGroups {{ sftp_grupa }}

# internal-sftp: SFTP poslužuje sam sshd, pa u chroot direktoriju ne trebaju programi ni biblioteke
Subsystem sftp internal-sftp

Match Group {{ sftp_grupa }}
  ChrootDirectory {{ sftp_root }}/%u
  ForceCommand internal-sftp -l INFO
  PermitTTY no
  AllowTcpForwarding no
  X11Forwarding no
```

yaml — Isječak 12 · s5/p4/roles/sftp/tasks/main.yml (izvod)

```yaml
- name: korisnici bez shella i bez lozinke (samo ključ)
  ansible.builtin.user:
    name: "{{ item.ime }}"
    group: "{{ sftp_grupa }}"
    shell: /usr/sbin/nologin
    home: /upload
    password: "*"                       # "*" = nema lozinke, ali račun nije zaključan ("!")
  loop: "{{ sftp_korisnici }}"

- name: chroot korijen po korisniku (root:root 0755)
  ansible.builtin.file: { path: "{{ sftp_root }}/{{ item.ime }}", state: directory, owner:
root, group: root, mode: "0755" }
  loop: "{{ sftp_korisnici }}"

- name: sshd_config iz šablona (sshd -t provjerava PRIJE zamjene)
  ansible.builtin.template:
    src: sshd_config.j2
    dest: "{{ sftp_etc }}/sshd_config"
    validate: /usr/sbin/sshd -t -f %s
  notify: restartuj sshd

- name: vatrozid iz šablona (nft -c provjerava PRIJE zamjene)
  ansible.builtin.template:
    src: sftp.nft.j2
    dest: "{{ sftp_etc }}/sftp.nft"
    validate: nft -c -f %s
  notify: učitaj vatrozid
```

Javni ključevi korisnika su u direktoriju koji pripada `root`-u (`AuthorizedKeysFile`), a ne u `~/.ssh`: korisnik ih ne može sam dopisati, pa je spisak dozvoljenih ključeva uvijek onaj iz koda. Ključ servera se pravi jednom; test od njega pravi `known_hosts`, pa klijent provjerava da razgovara s pravim serverom bez pitanja "yes/no". Vatrozid je isti obrazac kao u sedmici 4, samo na lancu `input` servera: `ip saddr { 10.50.0.1/32 } tcp dport 22 accept`, a sve ostalo na port 22 pada.

bash — Isječak 13 · pokretanje

```bash
make s5-p4          # ključevi korisnika, syntax-check i --check --diff (bez izmjena)
make s5-p4-apply    # opcionalno: rola dva puta (idempotentnost), pa test (podiže lab; traži
sudo)
make s5-p4-clean    # gasi sshd laba, briše korisnike i direktorije
```

izlaz 1. izvršavanja (PLAY RECAP)

```
localhost : ok=14 changed=11 unreachable=0 failed=0 skipped=1 rescued=0 ignored=0
```

izlaz 2. izvršavanja: changed=0 = idempotentnost

```
localhost : ok=12 changed=0 unreachable=0 failed=0 skipped=1 rescued=0 ignored=0
```

IDEMPOTENTNOST I SERVISI

Posljednja dva zadatka role provjeravaju da vatrozid i sshd stvarno rade, a `changed` javljaju samo ako su ih morali pokrenuti. Kad se lab ponovo podigne (namespace-ovi su novi, pa nema ni pravila ni procesa), prvo izvršavanje role javlja `changed=2` i vraća servis, a drugo ponovo `changed=0`. Rola tako opisuje željeno stanje, a ne samo korake instalacije.

izlaz testa (s5/p4/test.sh, skraćeno)

```
== 2. ruo-ana: sesija počinje u /upload; korijen (/) je chroot, a ne korijen servera
  sftp> pwd
  Remote working directory: /upload
  sftp> put izvjestaj.txt
  sftp> cd /
  sftp> ls -la
  drwxr-xr-x    ? 0        0            4096 Nov  6 16:45 .
  drwxr-xr-x    ? 0        0            4096 Nov  6 16:45 ..
  drwxr-x---    ? 30033    995          4096 Nov  6 16:46 upload
  ok: datoteka je na serveru u /srv/ruo5-sftp/ruo-ana/upload
  ok: u korijenu sesije postoji samo upload

== 3. ruo-ana: izvan chroota se ne vidi ništa, a u korijen chroota se ne može pisati
  File "/etc/passwd" not found.
  dest open "/izvjestaj.txt": Permission denied

== 5. Shell i lozinke nisu dozvoljeni
  $ ssh ruo-ana@10.50.0.22 id  ->  This service allows sftp connections only.
  $ ssh -o PreferredAuthentications=password ...  ->  ruo-ana@10.50.0.22: Permission denied
(publickey).

== 6. Vatrozid: iz ruo5-van (10.50.0.66) SSH port nije dostupan
  ruo5-van -> 10.50.0.22:22  blokiran
               ip saddr 10.50.0.1 tcp dport 22 counter packets 5 bytes 300 accept
               tcp dport 22 counter packets 3 bytes 180 drop comment "ssh iz nedozvoljene
mreže"

Sve provjere su prošle.
```

Za korisnika `/etc/passwd` "ne postoji": u njegovom korijenu je samo `upload`. Brojači pravila vatrozida pokazuju koliko je paketa pogodilo koje pravilo, pa se odmah vidi i dozvoljeni promet s računara i odbijeni pokušaji iz `ruo5-van`.

SAŽETAK I PROVJERA ZNANJA

Pregled naredbi, pitanja i zadaci

CILJ	ŠTA RADI	MIJENJA SISTEM?
`make s5-p1` / `s5-p1-up`	provjera konfiguracije i zona / lab, primarni i sekundarni DNS, dig upiti	ne / da (namespace-ovi, sudo)
`make s5-p1-serial`	izmjena zone bez i sa povećanjem seriala	da (radna kopija zone u labu)
`make s5-p2` / `s5-p2-apply`	Terraform: plan bez servera / apply, idempotentnost, drift i ispravak	ne / da (zapisi u `oblak.ruo.lab`)
`make s5-p3` / `s5-p3-up`	provjera konfiguracije pošte / servisi i test (relej, TLS, DKIM, SPF, DMARC)	ne / da (servisi u `ruo5-mail`)
`make s5-p4` / `s5-p4-apply`	SFTP rola: `--check` / dva izvršavanja + test	ne / da (2 lab korisnika, sshd u `ruo5-sftp`)
`make s5-clean`	briše sve iz sedmice 5 (servisi, korisnici, state, lab)	uklanja

PITANJE	ODGOVOR
Koja je razlika između autoritativnog i rekurzivnog DNS servera?	Autoritativni je izvor podataka za svoje zone; rekurzivni za klijenta prolazi lanac delegacija od korijena i pamti odgovore koliko kaže TTL.
Zašto sekundarni nije preuzeo izmjenu zone?	Serial u SOA zapisu nije povećan; sekundarni preuzima zonu samo kad je serial veći od njegovog.
Šta je glue zapis i kada je potreban?	A zapis za ime NS servera unutar zone koju taj server poslužuje; bez njega resolver ne bi mogao doći do servera.
Zašto server za `example.com` vraća REFUSED?	Server je samo autoritativni (`recursion no`) i ne odgovara za tuđe zone; otvoreni rekurzivni server se zloupotrebljava za DDoS.
Čemu služi TSIG ključ u primjeru 2?	Potpisuje DNS UPDATE, pa server prihvata izmjene samo od vlasnika ključa, i to samo u zoni koju mu dozvoljava `update-policy`.
Šta je otvoreni relej i koje pravilo ga sprječava?	Server koji prosljeđuje poštu bilo koga bilo kome; `reject_unauth_destination` dozvoljava tuđe domene samo prijavljenim korisnicima.
Šta provjerava SPF, a šta DKIM?	SPF: smije li IP adresa slati za domenu iz envelope pošiljaoca. DKIM: potpis domene nad zaglavljima i tijelom, pa i neizmijenjenost poruke.
Zašto treba DMARC ako postoje SPF i DKIM?	Tek DMARC traži da domena iz SPF-a ili DKIM-a odgovara domeni iz `From:` koju korisnik vidi, i kaže primaocu šta da uradi ako ne odgovara.
Zašto chroot direktorij mora pripadati root-u?	Inače bi korisnik u korijen mogao postaviti datoteke (npr. lažni `/etc`) i zloupotrijebiti programe koji se izvršavaju s višim pravima; sshd takvu konfiguraciju odbija.
Zašto su javni ključevi korisnika u direktoriju koji pripada root-u?	Korisnik ne može sam dodati ključ; spisak dozvoljenih ključeva je uvijek onaj iz koda.

Zadaci za samostalni rad

1. U zonu `ruo.lab` dodati zapis `git A 10.50.0.30` i odgovarajući PTR u reverznu zonu, povećati seriale, provjeriti obje zone (`make s5-p1`) i pokazati da odgovaraju oba servera.
2. Namjerno napraviti grešku u zoni (CNAME na `@` ili ime bez tačke na kraju) i predati izlaz `named-checkzone`. Objasniti poruku.
3. U `s5/p2/main.tf` dodati servis `cache = ["10.40.4.10"]` i `SRV` zapis za njega (`dns_srv_record_set`), primijeniti dva puta i predati izlaz drugog izvršavanja.
4. Promijeniti SPF zapis u `"v=spf1 -all"` i pokrenuti `mail.sh test`. Objasniti zašto SPF sada pada i za server `mail.ruo.lab`, a DMARC i dalje prolazi.
5. Napraviti drugi DKIM selektor (`ruo2027`), objaviti ga, prebaciti OpenDKIM na njega i opisati redoslijed zamjene ključa bez gubitka pošte.
6. U `vars.yml` dodati SFTP korisnika `ruo-cedo` i dozvoliti SSH iz `10.50.0.66/32`. Pokrenuti rolu dva puta i u `test.sh` dodati provjeru da je `ruo5-van` sada dozvoljen.

> **PROJEKTNI ZADATAK**
> Projekat traži DNS, mail i SFTP definisane kodom. Dobar početak je ova sedmica: zona domene projekta u Gitu, zapisi servisa iz Terraforma, mail server s potpunim SPF/DKIM/DMARC zapisima i SFTP korisnici iz Ansible podataka, uz test koji pokazuje šta radi, a šta je namjerno zabranjeno.

> **SLJEDEĆA SEDMICA**
> Servisi imaju imena i komuniciraju. Sedmica 6 uvodi podatke: block, file i object storage (NFS, MinIO/S3), baze podataka (PostgreSQL/MySQL) s korisnicima, privilegijama i replikacijom, te backup politike, RPO/RTO i test restore-a.